Early Prediction of Cache Thrashing in Changing Multicore Workloads

Herry Saraswat, Palash Mukherjee
Department of Electrical Engineering, Indian Institute of Science, Bengaluru, India
Modeling and Simulation – Phase 1 Project Proposal

Abstract— *A processor uses caches to keep recently used data close to its cores. Sometimes useful data is removed so often that the processor repeatedly fetches it again from slower memory. This project studies when that problem, called cache thrashing, appears in programs whose memory use changes over time. We will write small programs that access arrays in different ways, run them on simulated multicore computers using gem5, and record cache and performance measurements. We will compare warnings from a single measurement with warnings from the recent trajectory of several measurements. We will then estimate the chance of thrashing soon and test possible responses, chosen after examining the results.*

I. Introduction

A cache stores a small part of main memory so that a processor can use it quickly. If a program uses nearby data again, the cache can save time. If its active data becomes too large, or if several programs compete for the same cache space, useful data may be removed before it is reused. The processor then has to fetch it again. We call a continuing pattern of this kind *cache thrashing*.

A high cache miss rate on its own does not prove that thrashing occurred. For example, reading a large array once may cause many misses, but those values were not going to be reused. We are interested in cases where repeated cache misses are accompanied by a clear loss of performance compared with the same work on a computer with a sufficiently large cache.

Our question is: *Can we recognize warning signs of this slowdown early enough to reduce its impact?* Real programs do not always use memory in the same way. A task may scan an array, repeatedly use a small part of it, and later start accessing many scattered locations. The number of tasks running together can change too. We will test whether a recent sequence of cache and performance measurements gives a better warning than a prediction based on one measurement alone.

II. Related Work

Ding *et al.* showed how activity in an operating-system buffer can evict useful data belonging to other tasks from a shared last-level cache [1]. Their paper explains one specific cause of thrashing and tests a way to reduce it. Chandra *et al.* studied how threads competing for a shared cache affect one another's performance [2]. That work is relevant to our parallel workloads, although it predicts the effect of cache contention rather than the future start time of a thrashing episode.

Binkert *et al.* describe gem5, which lets us run the same workload on different simulated processor and cache designs [3]. Kleinbaum and Klein explain time-to-event analysis, including how to use runs that end before an event is observed [4]. Alcorta and Gerstlauer use performance counters to identify workload phases and predict changes in those phases [5]. Their prediction target is different from thrashing, but their use of recent measurements motivates our cache-state trajectory. Together, these works support our simulation setup and our estimate of near-future thrashing risk.

III. Brief Plan of Action

Step 1: Validate gem5. We will run small C++ examples with sequential, strided, random, and cache-set-conflicting array accesses. We will vary array size and stride and check simple cases by hand against gem5's reported misses. This will help us confirm that our chosen CPU and cache configuration behaves as expected before we run larger experiments.

Step 2: Generate workloads. Our changing workloads will be described by four inputs: working-set size (how much data is active), stride (distance between accesses), locality or reuse (how often data is revisited), and parallelism (number of active streams). A seeded generator will choose these settings and phase durations from stated distributions. A C++ program will then perform the resulting array reads and writes. We will save each generated phase sequence and reuse the same seed and sequence when comparing architectures. The four inputs control the *access pattern*; gem5 will measure the cache behavior that follows. Phases may increase pressure or allow recovery.

Step 3: Simulate several architectures. We will start with four cores, private caches and a shared last-level cache (LLC), then compare 1, 2, and 4 MiB LLCs while holding the other settings fixed. We will vary another setting, such as associativity, separately if time permits. In controlled runs, we will also change one workload input at a time to see its effect. Each run will produce measurements in windows of about 100,000 completed instructions, rather than one end-of-run average. These include misses per thousand instructions (MPKI), miss rate, evictions, completed instructions per cycle, and memory traffic. We will also run the same work with a much larger LLC as a reference. Independent seeds give different workload sequences; matched seeds allow fair comparisons between architectures.

Step 4: Define thrashing and estimate future risk. We will first conduct controlled simulation experiments using both known-thrashing and known-non-thrashing workloads. The known-thrashing cases will be constructed using mechanisms such as repeated accesses to more cache lines than the associativity of a cache set, pathological access strides, and working sets that repeatedly exceed the effective cache capacity, while known-non-thrashing cases will include cache-friendly and one-pass streaming workloads. For each experiment, we will collect time-windowed cache-state observations such as MPKI, miss rate, eviction rate, cache occupancy, re-reference behavior, memory traffic, and instruction throughput. We will compare the trajectories of these metrics between the controlled positive and negative cases to identify characteristics that distinguish

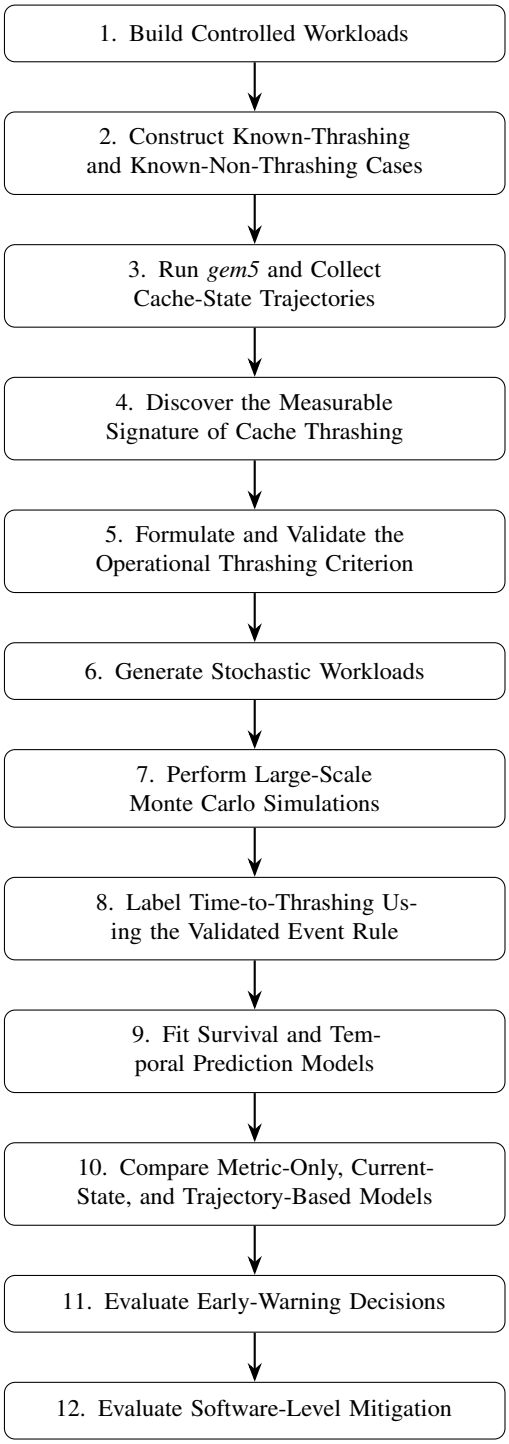


Figure 1: Overall workflow of the proposed cache-thrashing analysis framework.

genuine cache thrashing from a high cache-miss rate alone. Based on these observations, we will formulate an operational thrashing criterion using one or more measurable signals and validate it on previously unseen controlled workloads and parameter combinations. The event time T will be defined as the first observation window in which the validated criterion indicates the onset of a thrashing episode; cases in which no event occurs before the end of simulation will be treated as right-censored observations. This validated event definition will subsequently be used to label thrashing episodes in the large-scale stochastic workload and Monte Carlo experiments.

Step 5: Compare decisions and report results. Possible responses, such as temporarily reducing parallelism or changing task scheduling, will be selected after the first simulations; no action is the baseline. We will also compare acting only after an event with acting on an early warning. Each policy will finish the same amount of work on matched workloads. We will report event probability over time, warning accuracy and lead time, false alarms, total completion time, and the cost of unnecessary actions. Finally, we will compare which architecture handles each workload best and whether any tested action helps in that setting. The responses and their costs will be reported, even if doing nothing is best.

IV. Final Goal (Quantitative)

We aim to complete at least 30 random seeds for each of the three LLC sizes, giving at least 90 main runs, plus the large-cache reference runs and simple checks. These are planned experiments, not results. For correctly warned events, we hope for a median warning time of at least two windows, or about 200,000 completed instructions. For each response selected after the pilot, we will test whether it reduces extra completion time by at least 15% compared with responding after the event, while adding no more than 5% on runs with no event. These are evaluation targets, not promised improvements. We will report the actual results, uncertainty across seeds, and any target we do not reach. If simulation time limits the number of runs, we will state the smaller sample size clearly.

References

[1] X. Ding, K. Wang, and X. Zhang, "SRM-buffer: An OS buffer management technique to prevent last level cache from thrashing in multicores," in *Proc. EuroSys*, 2011.

[2] D. Chandra, F. Guo, S. Kim, and Y. Solihin, "Predicting inter-thread cache contention on a chip multi-processor architecture," in *Proc. HPCA*, 2005.

[3] N. Binkert *et al.*, "The gem5 simulator," *ACM SIGARCH Comput. Archit. News*, vol. 39, no. 2, 2011.

[4] D. G. Kleinbaum and M. Klein, *Survival Analysis: A Self-Learning Text*, 3rd ed. New York, NY, USA: Springer, 2012.

[5] E. S. Alcorta and A. Gerstlauer, "Learning-based workload phase classification and prediction using performance monitoring counters," in *Proc. MLCAD*, 2021.